

Regression Lens - CSC490 Project Proposal

Nuisance-Aware Semantic Visual Regression Detection for Web Interfaces

Group name: Regression Lens	Course: CSC490 - Machine Learning for Vision
Team: Abuzar Ansari, Abdullah Safi, Zehao Fan, Dev Vora	Repository: github.com/realdevvora/csc490-project

Machine-vision problem

We are addressing visual change detection for web interfaces. Given an expected (reference) screenshot and an actual rendered screenshot of the same webpage, the system should determine whether a difference is a meaningful regression, localize it, and classify what changed. This is harder than raw pixel comparison: browser rasterization, anti-aliasing, or other harmless rendering variation can change pixels without changing the intended interface, while a small shift in a button, font size, colour, alignment, or component state may be a real defect.

Name and location of datasets

Dataset	Location	Planned use
Design2Code	Hugging Face: SALT-NLP/Design2Code	Training-data generation from 484 webpages with source HTML and screenshots. We will create about 6-7 controlled mutations per webpage and render them with Playwright.
DiffSpot	Hugging Face: tencent/DiffSpot	Independent evaluation on paired screenshots with labelled fine-grained differences, no-change controls, mutation types, and difficulty tiers.

Reasons for choosing this problem and these datasets

This project represents a useful real-world application. Developers and QA engineers still spend substantial time visually comparing expected and actual interfaces. That work is repetitive, difficult to scale, and prone to missed regressions. All four team members have web-development experience and have encountered manual regression testing, so the problem is both familiar and practically motivating. Prior work on GUI visual testing, web inconsistency detection, cross-browser issues, and responsive-layout failures also shows that visual defects remain an important software-testing problem [3-7].

The project also presents interesting technical challenges. We must first create a reliable labelled training set: a script will modify Design2Code HTML, record the exact mutation, render the result with Playwright, and reject mutations that do not produce a useful visible change. Planned defect families reflect common QA triage cases: missing components or states, alignment/layout errors, typographic mismatches, colour-token violations, aspect-ratio distortion, icon/vector mismatches, and other appearance changes such as opacity, borders, or corner radius. Benign/placebo examples will let us test whether a model can distinguish harmless rendering noise from a true regression.

Finally, the dataset choice lets us explore domain generalization, an area of machine learning we want more experience with. We will train on synthetic-but-controlled mutations generated from Design2Code and evaluate on the independently built DiffSpot benchmark. DiffSpot contains 4,400 screenshot pairs: 3,900 true changes balanced across 13 CSS-property operators and three difficulty tiers, plus 500 no-difference controls [1]. Success therefore requires generalizing to webpages and visual changes not seen during training rather than memorizing layouts.

DiffSpot sample pairs

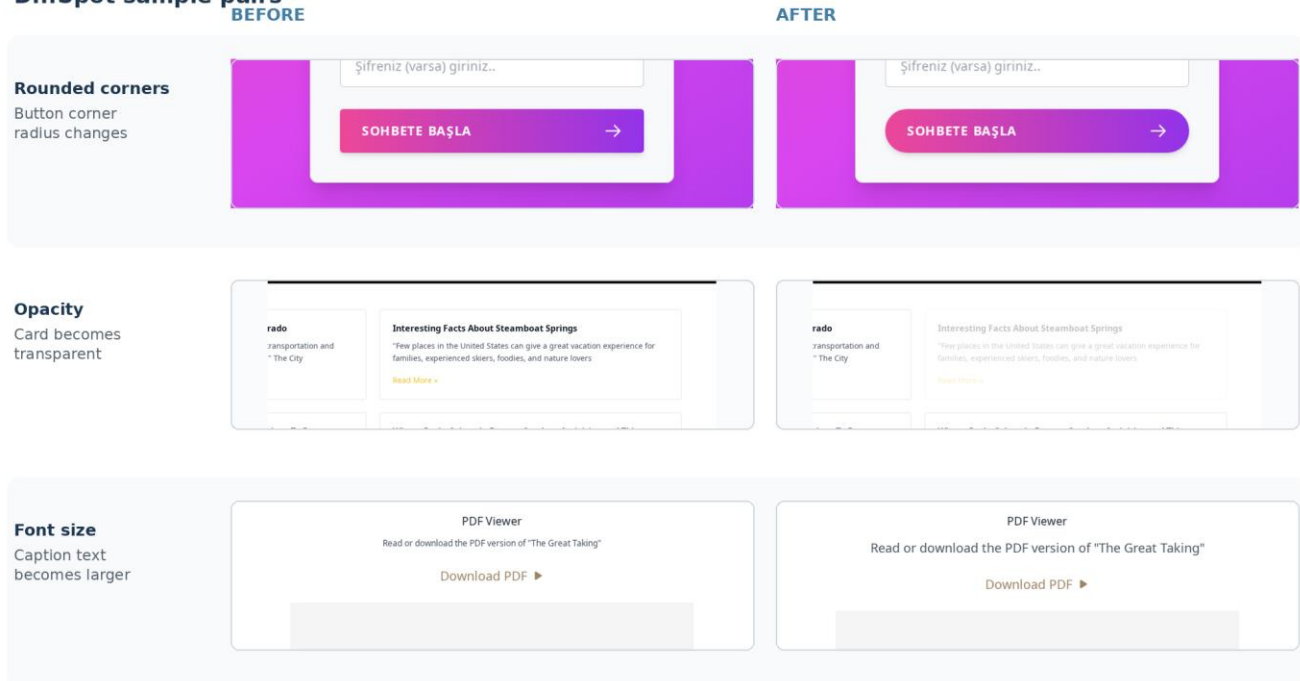


Figure 1. Cropped DiffSpot examples from the benchmark repository. Each before/after pair differs by one CSS-level mutation: rounded corners, opacity, or font size [1].

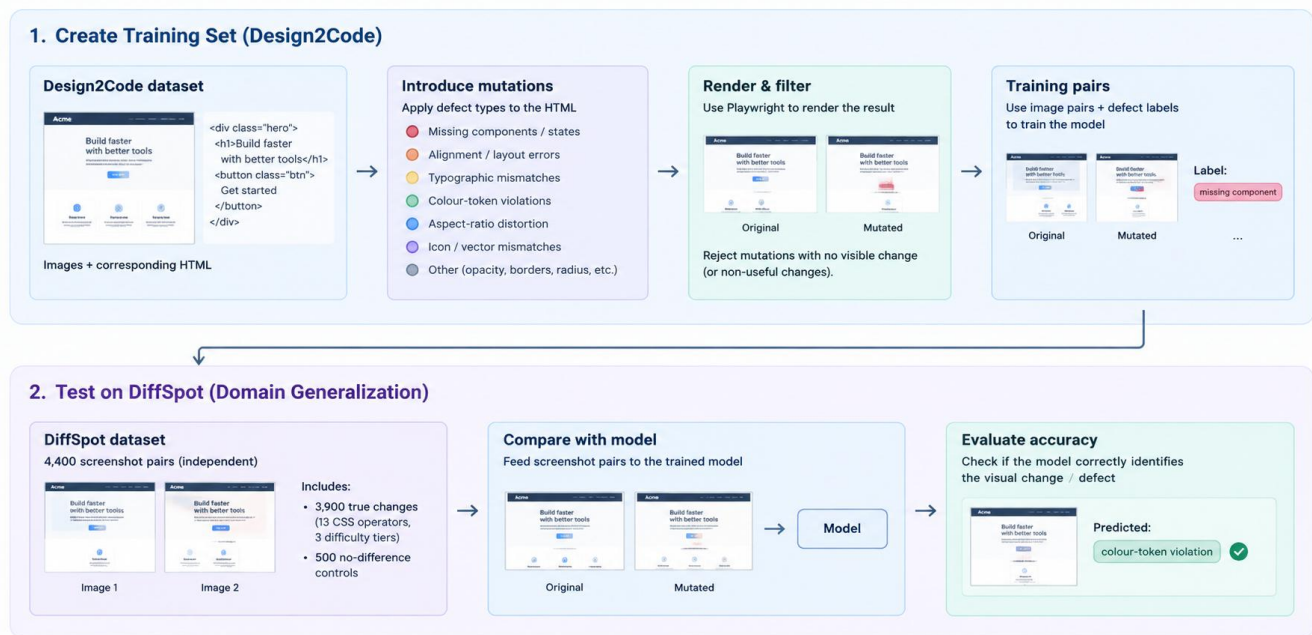


Figure 2. Tentative project pipeline: Design2Code HTML → mutation + Playwright render → candidate models → DiffSpot evaluation.

Distribution of responsibilities in the group (planned project)

Project stage	Responsibility
Data mutation / pre-training	All members: build the Design2Code mutation pipeline, generate about 6-7 labelled mutations per webpage, track benign vs. regression labels, and use Playwright to capture screenshots.
Model investigation	Each member investigates and trains one candidate model or model variation on the same generated dataset. Current directions include DINOv2, ChangeFormer, and variations of these models.
Testing	Each member evaluates their model on DiffSpot using the same test protocol, checking whether the model detects the change and whether the change is significant.
Model comparison / integration	The team compares all results using shared metrics and selects the strongest overall approach for the final presentation and further refinement.

How we plan to measure the accuracy of the system

We will evaluate both detection quality and generalization. The main measures are:

Measure	Metric / purpose
Regression detection	Precision, recall, and F1; also false-positive rate on benign/no-difference pairs.
Localization	Bounding-box IoU or pointing accuracy for the changed region.
Change category	Macro-F1 across planned QA defect classes and DiffSpot mutation types.
Generalization	Performance on DiffSpot after training only on generated Design2Code mutations.
Difficulty analysis	Recall/F1 split by DiffSpot Easy, Medium, and Hard tiers.

Distribution of tasks for this report

Member	Report contribution
Abuzar Ansari	Identified suitable datasets for the refined problem scope, helped define pipeline for data generation, training, and evaluation, and structured the report.
Abdullah Safi	Finalized the problem statement, handled the README, and wrote the motivation about why the project is personally interesting.
Zehao Fan	Contributed to the motivation sections on practical application, technical challenges, and the machine-learning knowledge the group wants to gain.
Dev Vora	Researched evaluation metrics and real-world visual-regression defect categories from prior literature and established the team's source-code repository.
Everyone	Worked together on the problem statement to ensure the group was aligned on the project scope and goals. All four team members have started accessing/downloading the datasets.

Scope note. This plan is intentionally tentative, as allowed by the assignment. If a model, mutation family, or evaluation task proves too difficult or too easy, we will narrow or extend the scope while preserving the core visual-regression question and shared evaluation protocol.

References

- [1] L. Zhang, A. Liu, Y. Liu, and X. Zhou, "DiffSpot: Can VLMs spot fine-grained visual differences in web interfaces?," arXiv preprint arXiv:2605.29615, 2026.
- [2] C. Si, Y. Zhang, R. Li, Z. Yang, R. Liu, and D. Yang, "Design2Code: Benchmarking multimodal code generation for automated front-end engineering," arXiv preprint arXiv:2403.03163, 2024.
- [3] A. Issa, J. Sillito, and V. Garousi, "Visual testing of graphical user interfaces: An exploratory study towards systematic definitions and approaches," in Proc. 14th IEEE Int. Symp. Web Syst. Evol. (WSE), 2012, pp. 11–15.
- [4] S. Mahajan, K. B. Gadde, A. Pasala, and W. G. J. Halfond, "Detecting and localizing visual inconsistencies in web applications," in Proc. 23rd Asia-Pacific Softw. Eng. Conf. (APSEC), 2016, pp. 361–364.
- [5] S. R. Choudhary, M. Prasad, and A. Orso, "X-PERT: Accurate identification of cross-browser issues in web applications," in Proc. 35th Int. Conf. Softw. Eng. (ICSE), 2013, pp. 702–711.
- [6] T. A. Walsh, G. M. Kapfhammer, and P. McMinn, "Automated layout failure detection for responsive web pages without an explicit oracle," in Proc. 26th ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA), 2017, pp. 192–203.
- [7] F. Macklon and C.-P. Bezemer, "Exploring the capabilities of vision-language models to detect visual bugs in HTML5 <canvas> applications," arXiv preprint arXiv:2501.09236, 2025.

Dataset and project links

Design2Code dataset: <https://huggingface.co/datasets/SALT-NLP/Design2Code>

DiffSpot dataset: <https://huggingface.co/datasets/tencent/DiffSpot>

DiffSpot code: <https://github.com/Tencent/DiffSpot>

Regression Lens repository: <https://github.com/realdevvora/csc490-project>